

Movie Recommendation System

Project Owner:

Aaron Irakoze

Michael S Barasa



Objective: This project aims to build a movie recommendation system based on user ratings and the genre.

Define the problem statement and importance

In the modern world, people are filled with many options to choose from because of prevalent data and internet. The recommender system helps people to filter data or information they require easily which helps to save time and energy. The movie recommendation system will help people to have manageable choices from the options of movies they would want to watch. This will be based on how the users (people who have scored the movie they previously watched) have rated the movie and the movies that have similar ratings on the same genre.

Explain the dataset (features and target variable)

Various used tools were used to complete this recommendation system such as:

Data analysis and manipulation

pandas- for loading and cleaning the data set also to control it.

numpy-for numerical operations and calculations.

For data visualization:

Seaborn- to add advanced and for colorful visualizations.

Matplotlib.pyplot- Creating visualizations like histograms which represent fixed representations of data.

Plotly.express- for creating interactive visualizations like histograms and bar charts displayed all around the output from code needed to create the system.

To create libraries for widgets and interactivity:

Ipywidgets- used for creating interactive widgets which are able to dropdown and show text inputs.

Ipython.display- for displaying widgets and clearing unwanted output.

Below shows the extracted data from kaggle by using imdb to use data from reviews, genre, votes, directors, countries (each movie released form), budget, gross and runtime and other more.

Dataset downloaded from [kaggle.com](https://www.kaggle.com)

```
iMovie_imdb.csv > data
1  name,rating,genre,year,released,score,votes,director,writer,star,country,budget,gross,company,runtime
2  The Shining,3.5,Drama,1980,"June 13, 1980 (United States)",8.4,927000,Stanley Kubrick,Stephen King,Jack Nicholson,United States
3  The Blue Lagoon,3.5,Adventure,1980,"July 2, 1980 (United States)",5.8,65000,Randal Kleiser,Henry De Vere Stacpoole,United States
4  Star Wars: Episode V - The Empire Strikes Back,3.5,Action,1980,"June 20, 1980 (United States)",8.7,1200000,Irvin Kershner,George Lucas,Mark Hamill,United States
5  Airplane!,3.5,Comedy,1980,"July 2, 1980 (United States)",7.7,221000,Jim Abrahams,Jim Abrahams,Robert Hays,United States
6  Caddyshack,3.5,Comedy,1980,"July 25, 1980 (United States)",7.3,108000,Harold Ramis,Brian Doyle-Murray,Chevy Chase,United States
7  Friday the 13th,3.5,Horror,1980,"May 9, 1980 (United States)",6.4,123000,Sean S. Cunningham,Victor Miller,Betsy Palmer,United States
8  The Blues Brothers,4,Action,1980,"June 20, 1980 (United States)",7.9,188000,John Landis,Dan Aykroyd,John Belushi,United States
9  Raging Bull,4,Biography,1980,"December 19, 1980 (United States)",8.2,330000,Martin Scorsese,Jake LaMotta,Robert De Niro,United States
10 Superman II,4,Action,1980,"June 19, 1981 (United States)",6.8,101000,Richard Lester,Jerry Siegel,Gene Hackman,United States
11 The Long Riders,4,Biography,1980,"May 16, 1980 (United States)",7.1,10000,Walter Hill,Bill Bryden,David Carradine,United States
12 Any Which Way You Can,4,Action,1980,"December 17, 1980 (United States)",6.1,18000,Buddy Van Horn,Stanford Sherman,Clay A. Thomas,United States
13 The Gods Must Be Crazy,4,Adventure,1980,"October 26, 1984 (United States)",7.3,54000,Jamie Uys,Jamie Uys,N!xau,South Africa
14 Popeye,4,Adventure,1980,"December 12, 1980 (United States)",5.3,30000,Robert Altman,Jules Feiffer,Robin Williams,United States
15 Ordinary People,3.5,Drama,1980,"September 19, 1980 (United States)",7.7,49000,Robert Redford,Judith Guest,Donald Sutherland,United States
16 Dressed to Kill,3.5,Crime,1980,"July 25, 1980 (United States)",7.1,37000,Brian De Palma,Brian De Palma,Michael Caine,United States
17 Somewhere in Time,4,Drama,1980,"October 3, 1980 (United States)",7.2,27000,Jeanne Szwarc,Richard Matheson,Christopher Reeve,United States
18 Fame,3.5,Drama,1980,"May 16, 1980 (United States)",6.6,21000,Alan Parker,Christopher Gore,Eddie Barth,United States
19 9 to 5,3.5,Comedy,1980,"December 19, 1980 (United States)",6.9,29000,Colin Higgins,Patricia Resnick,Jane Fonda,United States
20 The Fog,3,Horror,1980,"February 8, 1980 (United States)",6.8,66000,John Carpenter,John Carpenter,Adrienne Barbeau,United States
21 Stir Crazy,3.5,Comedy,1980,"December 12, 1980 (United States)",6.8,26000,Sidney Poitier,Bruce Jay Friedman,Gene Wilder,United States
22 Cruising,3.5,Crime,1980,"February 15, 1980 (United States)",6.5,20000,William Friedkin,William Friedkin,Al Pacino,United States
23 Heaven's Gate,3.5,Adventure,1980,"April 24, 1981 (United States)",6.8,14000,Michael Cimino,Michael Cimino,Kris Kristofferson,United States
24 The Final Countdown,4,Action,1980,"August 1, 1980 (United States)",6.7,22000,Don Taylor,Thomas Hunter,Kirk Douglas,United States
25 Xanadu,4,Fantasy,1980,"August 8, 1980 (United States)",5.3,12000,Robert Greenwald,Richard Christian Danus,Olivia Newton-John,United States
26 Urban Cowboy,3.5,Drama,1980,"June 6, 1980 (United States)",6.4,14000,James Bridges,Aaron Latham,John Travolta,United States
27 Altered States,3.5,Horror,1980,"December 25, 1980 (United States)",6.9,33000,Ken Russell,Paddy Chayefsky,William Hurt,United States
28 Little Darlings,4,Comedy,1980,"March 21, 1980 (United States)",6.5,5100,Ron Maxwell,Kimi Peck,Tatum O'Neal,United States
29 Raise the Titanic,4,Action,1980,"August 1, 1980 (United States)",5.4,100,Jerry Jameson,Adam Kennedy,Jason Robards,United States
30 Brubaker,3.5,Crime,1980,"June 20, 1980 (United States)",7.2,17000,Stuart Rosenberg,W.D. Richter,Robert Redford,United States
```

The given dataset shows key attributes for building a movie recommendation system which appears to be scattered (messy data), which also shows descriptive and categorical data about movies as their titles, ratings, genres, production details and financial information. Such attributes provide information for foundation for personalized recommendations based on what the user prefers. The code listed below shows 5 rows of the data set displayed.

```
movie = pd.read_csv('iMovie_imdb.csv')
movie.head()
```

The dataset contains columns such as:

(name, rating, genre, year, votes, director, budget and more)

Loading a csv file using pandas library

```
#csv file
import pandas as pd

movie = pd.read_csv('iMovie_imdb.csv')

movie.head()
```

✓ 0.0s

output

	name	rating	genre	year	released	score	votes	director	writer	star	country	budget	gross
0	The Shining	3.5	Drama	1980	June 13, 1980 (United States)	8.4	927000.0	Stanley Kubrick	Stephen King	Jack Nicholson	United Kingdom	19000000.0	46998772.0
1	The Blue Lagoon	3.5	Adventure	1980	July 2, 1980 (United States)	5.8	65000.0	Randal Kleiser	Henry De Vere Stacpoole	Brooke Shields	United States	4500000.0	58853106.0
2	Star Wars: Episode V - The Empire Strikes Back	3.5	Action	1980	June 20, 1980 (United States)	8.7	1200000.0	Irvin Kershner	Leigh Brackett	Mark Hamill	United States	18000000.0	538375067.0
3	Airplane!	3.5	Comedy	1980	July 2, 1980 (United States)	7.7	221000.0	Jim Abrahams	Jim Abrahams	Robert Hays	United States	3500000.0	83453539.0
4	Caddyshack	3.5	Comedy	1980	July 25, 1980 (United States)	7.3	108000.0	Harold Ramis	Brian Doyle-Murray	Chevy Chase	United States	6000000.0	39846344.0

Above shows 12 columns and 5 rows of imported movie data from kaggle directly form IMDB reviews of the movies used in the recommendation system.

The structure of the dataset

```
#dataset shape

movie.shape
```

✓ 0.0s

(7668, 15)

The movie.shape command line displays the structure of the dataset, which shows that the dataset contains 7669 rows and 15 columns

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from plotly.offline import init_notebook_mode
import seaborn as sns
import datetime as dt
import warnings
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', None)
init_notebook_mode(connected=True)

```

This then proceeds to load the dataset using:

Metplotlib, seaborn, plotly were used for the data visualization

The notebook includes library imports such as **pandas, numphy** which was used for handling the data,

Date time was used for date related processing for all the movies in the system.

`warnings.filterwarnings('ignore')`: minimizes warnings to provide a cleaner output

`pd.set_option('display.max_columns', None)`: this ensures that all columns are displayed when printing the finish touches for the data frames.

```

import re

def clean_name(name):
    title = re.sub('[^a-zA-Z0-9 ]', '', name)

movie["clean_name"] = movie["name"].apply(clean_name)

```

The code above shows the data cleaning process, by cleaning the movie names, the purpose for this is to remove special characters from the **name** columns and creates a new column **clean_name**.

Main purpose for this code is to help clean the movie names and remove special characters from the **name** and creates a column with a heading **clean_name**

```
from sklearn.model_selection import train_test_split
X = movie.drop(columns=['clean_name']) # Features
y = movie['clean_name'] # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In order to get the right data set for the movie recommendation system, we would have to split the data to get the accurate data set, this code splits the dataset into training and testing sets for machine learning tasks.

For the code listed below we see the numerical distribution, but when printed it produces the output showing the histograms for all the numerical columns, showing the distribution of values such as **rating**, **score**, **votes**, **budgets** and **gross**

```
movie.hist(figsize=(20, 20))
plt.show()
```

```
feature_distribution = make_subplots(rows=3, cols=3, subplot_titles=('<i>rating', '<i>genre', '<i>year',
feature_distribution.add_trace(go.Histogram(x=movie['rating'], name='rating'), row=1, col=1)
# Additional traces for other features...
feature_distribution.update_layout(template='plotly_dark', title_x=0.5)
feature_distribution.show()
```

In order to have the feature of distribution the given code below will print an output with not only just histograms but making them interactive for features such as **rating**, **genre**, **year**, and **score**.

The code below shows the input when we move the data frame **movie** be able to see the numerical distribution of all the numbers displayed in the columns.

```
#visualize numerical distribution

import matplotlib.pyplot as plt

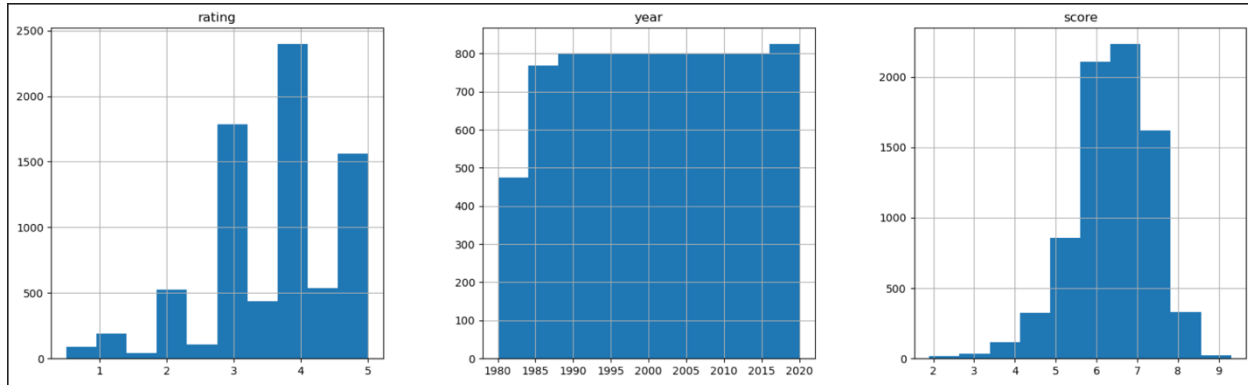
movie.hist(figsize=(20,20))
plt.show()

✓ 0.7s
```

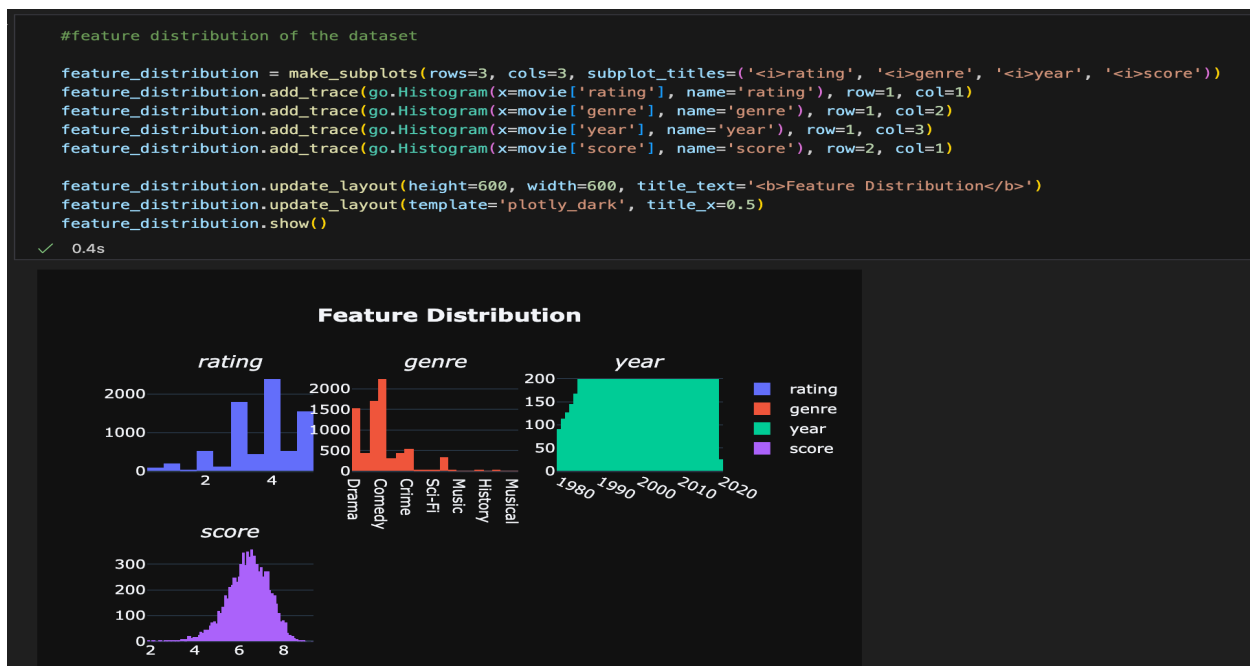
Now the output needs to be generated to show a grid of histograms created using **Matplotlib**, when this process is carried out each histogram will represent values for a numeric column in the

movie Data frame like given categories such as **score**, **rating** and other more, different axis's will represent different outputs such as x-axis will represent the range of values for each numeric column and Y will represents the frequency of values in the range.

Which will be displayed after running the output:



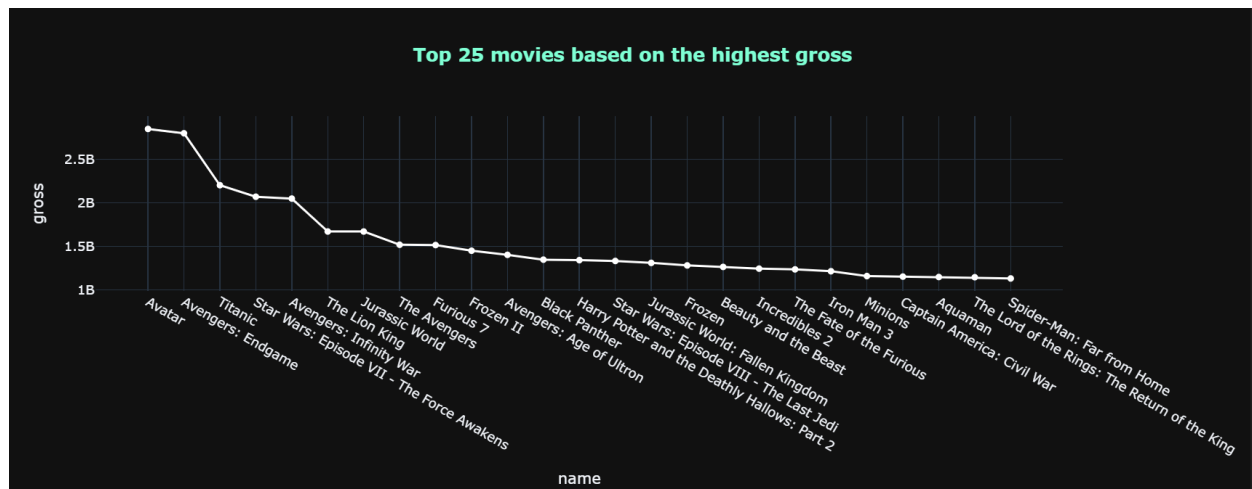
Since we need to know the total score on each genre of each movie in the recommendation system which is most popular, since we need to determine this rating we would need to use a histogram to show accurate results. The code listed below shows the input and output of the result:



After displaying the histogram, the code listed below shows the 25 movies, this helps by sorting the **movie** Data frame by the gross column in descending order, by selecting the top 25 movies with the highest gross revenue.

So in order to know which lands where we use the axis points such as **x= 'name'** (represents the names of 25 movies), **y= 'gross'** (represents the gross revenue of each movie).

The plot provides clear insights of comparison of the gross revenue of the top 25 movies, also allows users to see trends or patterns in the data, the feature also adds interactivity, enabling users to view more details such as director names.



After creating the output of histograms so that the display of the given data of the year, genre, year and score is displayed making it easier to identify which has the higher ratings between all 4 categories, we then convert the data on to numerical symbols using (TF-IDF) term frequency-inverse Document frequency. This also assists in converting movie names into numerical figures that can be used for machine learning models.

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(ngram_range=(1, 2))
tfidf = vectorizer.fit_transform(movie["name"])
```

Filtering dataset by dropping columns

This dataset contains 15 columns and only 6 are relevant for the final feature engineering and selection when building the widgets for searching a movie. In order to reduce the load of the dataset, we needed to drop other columns which is illustrated by the command line below:


```
# drop the columns that are not required in the feature engineering
movie.drop(columns=['released'], inplace=True)
movie.drop(columns=['director'], inplace=True)
movie.drop(columns=['writer'], inplace=True)
movie.drop(columns=['star'], inplace=True)
movie.drop(columns=['country'], inplace=True)
movie.drop(columns=['budget'], inplace=True)
movie.drop(columns=['gross'], inplace=True)
movie.drop(columns=['company'], inplace=True)
movie.drop(columns=['runtime'], inplace=True)
movie.drop(columns=['clean_name'], inplace=True)
```

✓ 0.0s

Feature engineering and selection

Feature Engineering and Selection

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(ngram_range=(1,2)) # Create a vectorizer object

# Create the TF-IDF matrix

tfidf = vectorizer.fit_transform(movie["name"])
```

✓ 0.0s

The command line above imports `TfidfVectorizer` from `sklearn.feature` module. `TfidfVectorizer` is used for conversion of a collection of text documents into a matrix of Term Frequency-Inverse Document Frequency features. The parameter `ngram_range=(1, 2)` is passed to the constructor which enables the vectorizer to consider both unigram and bigram. This is basically a machine learning model that enables to learn the data in the csv file.

Define search

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def search(name):
    # title = re.sub('[^a-zA-Z0-9 ]','', name) # Clean the input name
    query_vec = vectorizer.transform([name]) # Transform the input name into a vector
    similarity = cosine_similarity(query_vec, tfidf).flatten() # Calculate cosine similarity
    matched_movie = movie[movie["name"].str.lower() == name.lower()]
    target_genre= None
    target_rating=None
    if not matched_movie.empty:
        target_genre=matched_movie.iloc[0]['genre']
        target_rating=matched_movie.iloc[0]['rating']

    top_indices = np.argsort(similarity)[-50:]
    similar_movies = movie.iloc[top_indices].copy()
    similar_movies["similarity"] = similarity[top_indices]

    if target_genre and target_rating:
        similar_movies= similar_movies[
            similar_movies['genre'].str.contains(target_genre.split()[0], case=False, na=False)&(abs(similar_movies['rating']-target_rating)<= 1.0)
        ]

    similar_movies=similar_movies.sort_values(by='similarity', ascending=False).head(5)

    return similar_movies
```

✓ 0.0s

- The code defines a search() function that takes a movie name, transforms it into a vector using TfidfVectorizer.
- Calculates the cosine similarity between the input movie and a set of pre-processed movie data.
- It then finds the top 50 most similar movies based on cosine similarity and further filters these movies by genre and rating, if they match the input movie's genre and have a rating within 1 point of it.
- Finally, it returns the top 5 most similar movies, sorted by similarity, as the output.

Genre selection dropdown widget

```
import ipywidgets as widgets
from IPython.display import display, clear_output

# Step 1: Get unique genres for dropdown
unique_genres = sorted(set(g for g_list in movie["genre"].dropna() for g in g_list.split()))

# Step 2: Create dropdown widget
genre_dropdown = widgets.Dropdown(
    options=[" Select Genre "] + unique_genres,
    value=" Select Genre ",
    description="Genre:",
    disabled=False
)

# Step 3: Create output area
genre_output = widgets.Output()

# Step 4: Define the function that runs when dropdown changes
def on_genre_change(change):
    with genre_output:
        clear_output()
        selected_genre = change["new"]
        if selected_genre != "Select Genre":
            # Filter and sort movies by genre and rating
            filtered = movie[movie["genre"].str.contains(selected_genre, case=False, na=False)]
            sorted_movies = filtered.sort_values(by="rating", ascending=False)
            display(sorted_movies[["name", "genre", "rating", "year", "score", "votes"]].head(10))

# Step 5: Link dropdown to the function
genre_dropdown.observe(on_genre_change, names='value')

# Step 6: Display everything
display(genre_dropdown, genre_output)
```

✓ 0.0s

Genre: ▼

- This code creates an interactive dropdown widget in a Jupyter Notebook to filter and display movies based on their genre.
- First, it extracts unique genres from the movie dataset, sorts them, and creates a dropdown widget with those genres as options.
- When the genre is changed, the function `on_genre_change` filters the movies based on the selected genre, sorts them by rating, and displays the top 10 movies in a designate output area.

Interactive text input widget

```
import ipywidgets as widgets
from IPython.display import display

movie_input = widgets.Text(
    # value='Toy Story',
    placeholder='🔍 Enter keywords...',
    description='Movie Name:',
    disabled=False # False to enable the text box
)

movie_list = widgets.Output()

def on_type(data):
    with movie_list:
        movie_list.clear_output()
        Name = data["new"]
        if len(Name) > 3:
            display(search(Name))

movie_input.observe(on_type, names='value')

display(movie_input, movie_list)
```

Movie Name:

	name	rating	genre	year	score	votes	similarity
6444	Interstellar	5.0	Adventure	2014	8.6	1600000.0	1.0
2562	Jason's Lyric	3.0	Crime	1994	6.6	3800.0	0.0
2552	Intersection	5.0	Drama	1994	5.4	7900.0	0.0
2553	Sister My Sister	3.0	Drama	1994	6.6	3200.0	0.0
2554	Miracle on 34th Street	4.0	Family	1994	6.5	33000.0	0.0

- This code creates an interactive text input widget in a Jupyter Notebook that allows users to type a movie name or keywords.
- When the user types more than three characters, the `on_type` function is triggered, which calls the `search()` function to find and display movies related to the entered name.
- The results are displayed in an output area below the text box, updating dynamically as the user types.